

LÓGICA DE PROGRAMAÇÃO COM PYTHON

APRENDA A CRIAR PROGRAMAS QUE CONVERSAM COM OS USUÁRIOS



LÓGICA DE PROGRAMAÇÃO COM PYTHON

Caroline Perez Gonçalves

SUMÁRIO

APRESENTAÇÃO.....	1
INTRODUÇÃO	2
ENTRADA E SAÍDA DE DADOS	3
COMENTÁRIOS NO CÓDIGO.....	5
CONVERSÃO DE TIPOS (CASTING)	7
OPERADORES ARITMÉTICOS	9
OPERADORES RELACIONAIS	13
OPERADORES LÓGICOS	17
ESTRUTURAS CONDICIONAIS (IF, ELIF E ELSE)	20
PROJETO PRÁTICO	25
AGRADECIMENTO.....	29
CONTATO.....	30

APRESENTAÇÃO

Aos que não me conhecem, meu nome é Caroline. Sou formada em Análise e Desenvolvimento de Sistemas, pós-graduada em Inteligência Artificial e Machine Learning e possuo MBA em Gerenciamento de Projetos.

Além da minha experiência profissional com tecnologia, projetos e equipes multidisciplinares, também atuo como instrutora. Sempre acreditei que o conhecimento deve ser compartilhado de forma simples, leve e acessível, e foi com esse propósito que esta apostila foi desenvolvida.

O intuito deste material é mostrar que o Python está presente em muito mais situações do que imaginamos e que entender essa tecnologia não precisa ser algo complicado. Muito mais do que aprender ferramentas, o mais importante é compreender os conceitos, entender como a linguagem funciona, onde ela pode ser aplicada e como ela pode transformar as coisas.

Ao final desta apostila, espero que você tenha construído uma base sólida sobre os fundamentos dessa linguagem e se sinta mais confiante para acompanhar as próximas trilhas de estudo e explorar esse universo com mais segurança.

Vamos aos estudos! Mas, antes, sente-se, relaxe e sinta-se em casa. Quero que este material seja como uma boa conversa entre amigos, tornando o aprendizado mais leve, agradável e, principalmente, significativo.

Boa sorte e muito sucesso!

INTRODUÇÃO

Afinal, por que aprender Python?

A resposta para essa e outras perguntas tem um segredo. Muita gente acredita que programação é um assunto complicado, cheio de códigos difíceis ou exclusivo para quem trabalha com tecnologia. Mas a verdade é justamente o contrário.

Como assim, Carol?

A programação está presente em praticamente tudo o que fazemos. Ela está por trás dos aplicativos que usamos diariamente, dos sites que acessamos, dos sistemas bancários, das compras online, dos aplicativos de transporte, das redes sociais e até de muitos equipamentos que fazem parte da nossa rotina.

Quando você entende isso, percebe que aprender Python não significa apenas estudar uma linguagem de programação. Significa desenvolver uma nova forma de pensar, aprender a resolver problemas de maneira lógica e criar soluções para situações do dia a dia.

E tem mais: aprender Python não é decorar comandos. Os comandos podem até mudar de uma linguagem para outra, mas o que realmente faz diferença é compreender os conceitos, desenvolver o raciocínio lógico e entender como construir programas capazes de automatizar tarefas e facilitar o trabalho das pessoas.

Hoje o Python é utilizado em diversas áreas, como desenvolvimento de sistemas, automação, análise de dados, Inteligência Artificial, desenvolvimento Web, computação em nuvem, segurança da informação e muitas outras. Isso faz dele uma das linguagens mais versáteis e procuradas do mercado.

Claro, existem assuntos mais avançados que exigem bastante estudo e prática. Mas, como em qualquer aprendizado, tudo começa pelos fundamentos. E é exatamente isso que vamos construir ao longo desta apostila.

Para que essa introdução não fique muito extensa, vamos explorar cada tema no momento certo. E, como todo bom aprendizado, vamos começar pelo começo.

Vamos lá.

ENTRADA E SAÍDA DE DADOS

Na apostila anterior aprendemos a criar variáveis e armazenar informações.

Mas existe um problema.

Até agora, todas as informações foram escritas diretamente no código. Por exemplo.

```
nome = "Caroline"
```

Isso funciona. Mas imagine um sistema de cadastro de clientes. Será que faz sentido alterar o código toda vez que um novo cliente chegar?

Claro que não.

Seria praticamente impossível utilizar um programa dessa forma. É justamente por isso que existem os comandos de **entrada e saída de dados**.

Eles permitem que o programa converse com quem está utilizando o sistema. Em outras palavras. O programa faz uma pergunta e o usuário responde. E o programa utiliza essa resposta para continuar executando suas tarefas.

É exatamente assim que funciona a maioria dos sistemas que utilizamos diariamente.

Quando você faz login em um aplicativo, por exemplo, acontece algo parecido. O sistema pede seu e-mail. Depois solicita sua senha. Você informa esses dados e o programa decide o que fazer com essas informações. Percebe?

Você está interagindo com o sistema.

Em Python, essa interação acontece principalmente através de duas funções.

```
input();
```

```
print().
```

Vamos conhecer cada uma delas.

Exibindo informações com print()

Sempre que quisermos mostrar alguma informação na tela, utilizaremos a função **print()**. Veja um exemplo.

```
print("Bem-vindo ao sistema!")
```

O resultado será:

```
Bem-vindo ao sistema!
```

Parece simples. E realmente é. Mas praticamente todos os programas utilizam essa função em algum momento. Podemos utilizá-la para:

- Exibir mensagens ao usuário;

- Mostrar resultados de cálculos;
- Informar erros;
- Confirmar operações;
- Apresentar menus;
- Exibir informações armazenadas em variáveis.

Por exemplo.

```
nome = "Caroline"
```

```
print(nome)
```

Resultado: Caroline

Observe que, neste caso, não utilizamos aspas dentro do `print()`. Isso acontece porque queremos mostrar o conteúdo da variável, e não o texto "nome".

Recebendo informações com `input()`

Agora imagine que queremos descobrir o nome do usuário. Não faz sentido escrever o nome diretamente no código, já que cada pessoa que utilizar o programa terá um nome diferente.

Para resolver esse problema utilizamos a função **`input()`**.

Ela permite que o programa aguarde uma resposta do usuário. Veja um exemplo.

```
nome = input("Digite seu nome: ")
```

Quando o programa for executado, aparecerá algo parecido com isto.

Digite seu nome:

O usuário poderá digitar Carlos, por exemplo. Depois de pressionar Enter, essa informação será armazenada na variável **nome**. A partir desse momento podemos utilizá-la normalmente.

```
print(nome)
```

Resultado: Carlos

Perceba como o programa começa a ficar mais interessante. Agora ele já consegue receber informações de quem está utilizando o sistema. Informações reais, que se são recebidas, armazenadas e utilizadas quando necessário.

Com um simples comando, já podemos ter algo incrível funcionando.

Juntando input() e print()

Na programação, é muito comum receber uma informação e depois utilizá-la em outra mensagem. Veja um exemplo.

```
nome = input("Digite seu nome: ")  
  
print("Olá,", nome)
```

Se o usuário digitar Fernanda

O resultado será: Olá, Fernanda

Parece uma coisa simples. Mas é exatamente esse tipo de interação que acontece em milhares de sistemas. Quando você faz uma compra online, quando acessa um banco, quando entra em uma rede social ou quando agenda uma consulta.

Em todos esses casos, o sistema faz perguntas, recebe respostas e apresenta novas informações ao usuário.

Um detalhe importante

Quando utilizamos a função input(), o Python sempre recebe a informação como **texto** mesmo que o usuário digite apenas números. Por exemplo.

Se o usuário informar 25, o Python armazenará: "25"

Ou seja, um texto.

Mais adiante aprenderemos como transformar esse texto em números para realizar cálculos. Esse processo recebe o nome de **conversão de tipos**, e será um dos próximos assuntos desta apostila, hein?!

COMENTÁRIOS NO CÓDIGO

Quando começamos a programar, normalmente lembramos de tudo o que fizemos. Mas imagine só a seguinte situação:

Você desenvolveu um programa hoje. Seis meses depois, precisa voltar naquele mesmo código para fazer uma alteração. Será que ainda vai lembrar exatamente por que escreveu determinada parte?

Provavelmente não.

Agora imagine um cenário ainda mais comum. Você faz parte de uma equipe de desenvolvimento e outro programador precisa dar continuidade ao seu trabalho. Como ele vai entender o que você fez?

É justamente para isso que existem os **comentários**.

Os comentários servem para explicar trechos do código, registrar informações importantes ou fazer pequenas anotações para quem irá ler aquele programa no futuro. E atente-se em como vai deixar o comentário, ok? Nada de deixar algo sem sentido.

O mais interessante é que o Python ignora completamente essas linhas. Ou seja, elas existem apenas para facilitar a leitura do código.

Para criar um comentário em Python utilizamos o símbolo `#`. Veja um exemplo.

```
# Cadastro do cliente
```

```
nome = input("Digite seu nome: ")
```

Tudo o que estiver depois do símbolo `#` será considerado um comentário. No caso, a linha `# Cadastro do cliente`. Podemos utilizá-los para diversas finalidades.

- Explicar uma parte mais complexa do código;
- Organizar o programa em seções;
- Registrar observações importantes;
- Facilitar a manutenção do sistema.

Mas existe um cuidado importante. Como disse, os comentários devem ajudar e não atrapalhar. Veja este exemplo.

```
# Criando uma variável chamada nome
```

```
nome = input("Digite seu nome: ")
```

Esse comentário praticamente repete aquilo que o próprio código já mostra. Já sabemos que criamos uma variável para receber entrada de dados. Agora observe este outro.

```
# Recebe o nome que será utilizado no cadastro do cliente
```

```
nome = input("Digite seu nome: ")
```

Muito melhor. Nesse caso o comentário explica o objetivo daquela informação. Sempre que possível, escreva comentários que agreguem valor.

Outra dica importante: um código bem escrito costuma precisar de poucos comentários. Quando escolhemos bons nomes para as variáveis e organizamos corretamente o programa, grande parte do código já se torna autoexplicativo, sem necessidade de mais linhas ou detalhes. Escreva seu código e leia quantas vezes forem necessárias para ter certeza que foi feito um bom desenvolvimento. De maneira clara e limpa.

Use comentários quando realmente houver necessidade. Eles são uma excelente ferramenta, mas não substituem um código organizado.

CONVERSÃO DE TIPOS (CASTING)

Já aprendemos a utilizar a função **input()**.

Ela permite que o usuário informe dados para o programa. Mas existe um detalhe muito importante que todo iniciante precisa conhecer.

O Python sempre interpreta o valor digitado como um **texto**.

Lembra disso? Mesmo quando o usuário digita apenas números. Veja este exemplo.

```
idade = input("Digite sua idade: ")  
  
print(idade)
```

Se o usuário digitar:

25

O Python armazenará esse valor como texto. E isso pode gerar um problema. Imagine que queremos somar a idade do usuário com mais um ano.

Veja o código.

```
idade = input("Digite sua idade: ")  
  
print(idade + 1)
```

Esse programa não funcionará corretamente. Por quê?

Porque não podemos somar um texto com um número inteiro. É nesse momento que entra o **casting**, também chamado de **conversão de tipos**. O casting consiste em transformar um tipo de dado em outro.

Em Python, as conversões mais utilizadas são:

- `int()`;
- `float()`;
- `str()`;
- `bool()`.

Vamos conhecer cada uma delas.

Convertendo para inteiro

Vamos ver as conversões logo abaixo com trechos de códigos simples.

Quando queremos transformar um texto em um número inteiro, utilizamos **int()**. Veja um exemplo.

```
idade = int(input("Digite sua idade: "))  
  
print(idade + 1)
```

Agora sim. Se o usuário informar **25**, o Python entenderá que esse valor representa um número.

O resultado será: 26

Convertendo para decimal

Imagine um sistema que calcula o valor de uma compra. O usuário informa:

189.90

Como esse número possui casas decimais, utilizamos **float()**.

```
valor = float(input("Digite o valor da compra: "))
```

Agora o programa poderá realizar cálculos normalmente.

Convertendo para texto

Em algumas situações também precisamos transformar números em texto.

Para isso utilizamos **str()**.Veja um exemplo.

```
numero_pedido = 1250
```

```
mensagem = "Número do pedido: " + str(numero_pedido)
```

```
print(mensagem)
```

O resultado será:

Número do pedido: 1250

Convertendo para booleano

Também é possível utilizar a função **bool()**.

Ela converte um valor para o tipo booleano (True ou False), seguindo as regras definidas pelo Python. Embora esse tipo de conversão exista, ela é menos utilizada por quem está começando.

Ao longo da trilha teremos oportunidade de utilizá-la quando fizer mais sentido. Por enquanto, basta saber que ela também faz parte das funções de conversão do Python.

Quando utilizar o casting?

Sempre que houver necessidade de trabalhar com um tipo de dado diferente daquele que está armazenado. Alguns exemplos são esse que foram apresentados acima.

Você perceberá que praticamente todo sistema utiliza casting em algum momento.

OPERADORES ARITMÉTICOS

Até agora, aprendemos a armazenar informações em variáveis e receber dados do usuário. Mas imagine uma outra situação:.

Você foi contratado para desenvolver um sistema para uma loja. Sempre que um cliente finalizar uma compra, será necessário calcular:

- O valor total da compra;
- O desconto aplicado;
- O valor do frete;
- O valor final do pedido.

Como o programa faria esses cálculos?

É justamente para isso que existem os **operadores aritméticos**. Eles permitem realizar operações matemáticas utilizando números armazenados em variáveis.

Embora o nome pareça complicado, provavelmente você já conhece todos eles. São os mesmos operadores utilizados nas contas que fazemos diariamente. Vamos conhecer cada um deles.

Adição (+)

O operador de adição é utilizado para somar valores.

Veja um exemplo.

```
valor_produto = 150
```

```
frete = 20
```

```
total = valor_produto + frete
```

```
print(total)
```

Resultado: 170

A soma é uma das operações mais utilizadas em qualquer sistema. Ela aparece em situações como a soma de valor de produtos, total de horas trabalhadas, quantidade de itens vendidos, etc.

Subtração (-)

O operador de subtração é utilizado para calcular diferenças. Veja um exemplo.

```
saldo = 1000
```

```
saque = 250
```

```
saldo = saldo - saque
```

```
print(saldo)
```

Resultado: 750

Esse operador aparece frequentemente em situações como desconto em compras, baixa de estoque, saques bancários, etc.

Multiplicação (*)

A multiplicação é utilizada quando precisamos repetir ou combinar valores. Veja um exemplo.

```
preco = 80
```

```
quantidade = 3
```

```
total = preco * quantidade
```

```
print(total)
```

Resultado: 240

Esse cálculo é muito comum em sistemas comerciais. Sempre que compramos vários produtos iguais, a multiplicação entra em ação.

Divisão (/)

O operador de divisão é utilizado para dividir um valor por outro. Veja um exemplo.

```
valor_total = 300
```

```
parcelas = 3
```

```
valor_parcela = valor_total / parcelas
```

```
print(valor_parcela)
```

Resultado: 100.0

Observe um detalhe importante...

Mesmo quando a divisão resulta em um número inteiro, o Python retorna um valor decimal. Isso acontece porque o operador / sempre produz um resultado do tipo **float**.

Divisão inteira (//)

Em algumas situações não queremos o valor decimal. Queremos apenas a parte inteira da divisão. Veja um exemplo.

```
clientes = 11
```

```
mesas = 2
```

```
resultado = clientes // mesas
```

```
print(resultado)
```

Resultado: 5

A parte decimal é descartada. Esse operador é bastante utilizado quando precisamos trabalhar apenas com números inteiros.

Resto da divisão (%)

Imagine que você queira descobrir se um número é par ou ímpar. Ou verificar se ainda restaram produtos após dividir um lote. Para isso utilizamos o operador de resto da divisão. Veja um exemplo.

```
resultado = 15 % 2
```

```
print(resultado)
```

Resultado: 1

Isso significa que:

15 dividido por 2 é igual a 7, restando 1.

Esse operador aparece bastante em sistemas reais, como quando precisamos descobrir se um número é par, verificar ciclos, trabalhar com datas ou distribuir tarefas automaticamente.

Potenciação ()**

Também podemos calcular potências utilizando Python. Veja um exemplo.

```
resultado = 5 ** 2
```

```
print(resultado)
```

Resultado: 25

Nesse caso estamos calculando 5^2 .

Outro exemplo.

```
resultado = 2 ** 10
```

```
print(resultado)
```

Resultado: 1024

Resumindo os operadores para ficar mais direto:

Os operadores aritméticos mais utilizados são:

+ → Adição;

- → Subtração;

* → Multiplicação;

/ → Divisão;

// → Divisão inteira;

% → Resto da divisão;

** → Potenciação.

Pode parecer bastante informação. Mas, com a prática, esses operadores passam a fazer parte do dia a dia de qualquer programador.

Um detalhe importante

Assim como na matemática, o Python também segue uma ordem para realizar os cálculos. Veja este exemplo.

```
resultado = 10 + 5 * 2
```

```
print(resultado)
```

O resultado será 20 e não 30.

Isso acontece porque a multiplicação é realizada antes da soma. Se quisermos alterar essa ordem, utilizamos parênteses.

```
resultado = (10 + 5) * 2
```

```
print(resultado)
```

Resultado: 30

Sempre que houver dúvidas sobre a ordem das operações, utilize parênteses. Além de evitar erros, eles deixam o código muito mais fácil de entender.

OPERADORES RELACIONAIS

Vamos imaginar outra situação? Já vimos como receber informações do usuário e a realizar cálculos. Mas e se você precisa desenvolver um sistema para uma locadora de veículos?

Antes de liberar um carro, o sistema precisa verificar:

- O motorista possui mais de 18 anos?
- A carteira de habilitação está válida?
- Existe algum débito em aberto?

Ou imagine um sistema de uma loja. Antes de concluir uma venda, ele precisa verificar:

- Existe quantidade suficiente em estoque?
- O pagamento foi aprovado?
- O desconto informado é válido?

Percebe uma coisa? Nesses casos o programa não precisa fazer contas. Ele precisa **comparar informações**.

É exatamente para isso que existem os **operadores relacionais**. Eles permitem verificar se uma determinada condição é verdadeira ou falsa. Na maioria das vezes, o resultado será apenas um destes dois valores.

- True;
- False.

Esses operadores serão fundamentais para o próximo assunto da apostila, onde aprenderemos a criar programas capazes de tomar decisões.

Vamos conhecer cada um deles igual fizemos com os operadores aritméticos.

Igual a (==)

O operador == verifica se dois valores são iguais. Veja um exemplo.


```
idade = 18
```

```
print(idade == 18)
```

Resultado: True

Agora observe outro exemplo.

```
idade = 22
```

```
print(idade == 18)
```

Resultado: False

Perceba um detalhe importante. Não confunda = com ==.

O sinal de igual (=) é utilizado para atribuir um valor a uma variável.

Já o operador == é utilizado para comparar dois valores.

Essa é uma das dúvidas mais comuns entre quem está começando a programar.

Diferente de (!=)

O operador != verifica se dois valores são diferentes. Veja um exemplo.

```
status = "Ativo"
```

```
print(status != "Inativo")
```

Resultado: True

Agora observe outro exemplo.

```
status = "Inativo"
```

```
print(status != "Inativo")
```

Resultado: False

Esse operador é bastante utilizado para verificar situações em que um determinado valor não pode ser igual ao esperado.

Maior que (>)

O operador > verifica se um valor é maior que outro. Veja um exemplo.

```
idade = 25
```

```
print(idade > 18)
```

Resultado: True

Esse tipo de comparação aparece em diversas situações.

- Verificar idade mínima;
- Conferir limite de crédito;
- Validar quantidade em estoque;
- Avaliar metas de vendas.

Menor que (<)

O operador < verifica se um valor é menor que outro. Veja um exemplo.

```
temperatura = 12
```

```
print(temperatura < 18)
```

Resultado: True

Também é bastante utilizado em sistemas de monitoramento, controle de estoque e validação de dados.

Maior ou igual (>=)

Às vezes não basta saber se um valor é maior. Também precisamos aceitar quando ele for exatamente igual. Veja um exemplo.

```
idade = 18
```

```
print(idade >= 18)
```

Resultado: True

Imagine um sistema que permita o acesso apenas para maiores de idade. Quem possui exatamente 18 anos também deve ser autorizado. É justamente para isso que utilizamos >=.

Menor ou igual (<=)

O operador <= funciona de forma semelhante. Ele verifica se um valor é menor ou igual ao valor informado. Veja um exemplo.

```
velocidade = 80
```

```
print(velocidade <= 80)
```

Resultado: True

Esse tipo de comparação é bastante comum em regras de negócio.

- Limite máximo de velocidade;
- Quantidade máxima de participantes;
- Valor máximo permitido;
- Peso máximo suportado.

Comparando textos

Os operadores relacionais não servem apenas para números. Também podemos comparar textos. Veja um exemplo.

```
usuario = "admin"
```

```
print(usuario == "admin")
```

Resultado: True

Outro exemplo.

```
cidade = "Curitiba"
```

```
print(cidade != "São Paulo")
```

Resultado: True

Isso permite validar usuários, perfis de acesso, cidades, estados, categorias e diversas outras informações.

O resultado sempre será verdadeiro ou falso

Independentemente da comparação realizada, o resultado será sempre um valor booleano. Ou seja, True ou False.

É justamente esse resultado que permitirá ao programa decidir qual caminho seguir.

Mais adiante veremos que uma estrutura como o `if` utiliza exatamente esse tipo de resposta. Pense da seguinte forma.

O operador relacional faz uma pergunta. O Python responde apenas de duas maneiras: `sim` ou `não`. Ou, tecnicamente falando, `true` ou `false`.

OPERADORES LÓGICOS

É uma apostila de Python, mas estamos também trabalhando a imaginação. Vamos, agora, imaginar uma situação mais complexa. Você ainda está desenvolvendo um sistema para uma locadora de veículos.

Para que o aluguel seja aprovado, não basta apenas verificar a idade do cliente. Também será necessário confirmar se:

- O cliente possui 18 anos ou mais;
- A Carteira Nacional de Habilitação (CNH) está válida.

Percebe? Agora temos **duas condições**.

Imagine outro exemplo.

Um sistema de uma loja oferece frete grátis quando:

- A compra for maior que R\$ 300,00;
- O cliente morar na mesma cidade da loja.

Mais uma vez, duas condições precisam ser analisadas. É justamente para resolver situações como essas que existem os **operadores lógicos**.

Eles permitem combinar duas ou mais condições em uma única expressão. Em Python, os três operadores lógicos mais utilizados são:

- `and`;
- `or`;
- `not`.

Vamos conhecer cada um deles.

Operador AND

O operador **`and`** significa **"e"**. Ele exige que **todas** as condições sejam verdadeiras.

Veja um exemplo.

```
idade = 22
```

```
cnh_valida = True
```

```
print(idade >= 18 and cnh_valida)
```

Resultado: True

Agora imagine que a CNH esteja vencida.

```
idade = 22
```

```
cnh_valida = False
```

```
print(idade >= 18 and cnh_valida)
```

Resultado: False

Mesmo que a idade esteja correta, o resultado será falso.

Isso acontece porque o operador **and** exige que todas as condições sejam verdadeiras ao mesmo tempo.

Pense da seguinte forma:

Imagine que uma empresa só libera o acesso ao prédio quando o funcionário apresenta:

- Crachá;
- Documento com foto.

Se faltar qualquer um deles, a entrada será negada. É exatamente assim que funciona o operador **and**.

Operador OR

O operador **or** significa "**ou**". Nesse caso, basta que **uma** das condições seja verdadeira.

Veja um exemplo.

```
cliente_vip = False
```

```
cupom_desconto = True
```

```
print(cliente_vip or cupom_desconto)
```

Resultado: True

Mesmo que o cliente não seja VIP, ele possui um cupom de desconto. Como uma das condições foi satisfeita, o resultado será verdadeiro.

Outro exemplo.

Imagine um estacionamento. A entrada é permitida para:

- Clientes;
- Funcionários.

Se a pessoa se enquadrar em qualquer uma dessas situações, ela poderá entrar. Esse é o comportamento do operador **or**.

Operador NOT

O operador **not** funciona de maneira diferente. Ele inverte o valor de uma condição.

Veja um exemplo.

```
pedido_cancelado = False  
print(not pedido_cancelado)
```

Resultado: True

Perceba o que aconteceu. O valor original era **False**. Após utilizar o operador **not**, ele passou a ser **True**.

Outro exemplo.

```
usuario_logado = True  
print(not usuario_logado)
```

Resultado: False

Sempre que utilizamos o operador **not**, o Python faz exatamente o contrário da condição informada.

Utilizando vários operadores juntos

Em programas reais é muito comum combinar diversos operadores.

Imagine um sistema de um banco. Para aprovar um financiamento, o cliente precisa:

- Possuir renda suficiente;
- Não possuir restrições no CPF.

Veja um exemplo.

```
renda = 6500  
cpf_regular = True
```

```
print(renda >= 5000 and cpf_regular)
```

Resultado: True

Outro exemplo.

Uma plataforma de streaming oferece acesso ao conteúdo para:

- Usuários Premium;
- Usuários que compraram o filme separadamente.

```
usuario_premium = False
```

```
filme_comprado = True
```

```
print(usuario_premium or filme_comprado)
```

Resultado: True

Essas combinações aparecem o tempo todo em sistemas reais.

E quer saber uma coisa engraçada? Eu tive dificuldade em entender isso quando comecei a faculdade. Apesar de ser simples, parece um bicho de 10 cabeças quando olha pela primeira vez essas informações em uma tabela. Mas, saiba que não é. Hoje, enquanto digito isso, a minha única preocupação, foi ler e tentar escrever da melhor forma possível para não ficar confuso. Eu pensei várias vezes: está simples demais e ninguém vai entender.

Resumindo

Os principais operadores lógicos do Python são:

- and → Todas as condições precisam ser verdadeiras;
- or → Basta que uma das condições seja verdadeira;
- not → Inverte o resultado de uma condição.

Pode parecer simples.

ESTRUTURAS CONDICIONAIS (IF, ELIF E ELSE)

Até aqui, nossos programas já conseguem fazer bastante coisa. Eles conseguem:

- Receber informações do usuário;
- Exibir mensagens na tela;
- Realizar cálculos;

- Comparar valores;
- Combinar diferentes condições.

Mas ainda existe uma limitação. Imagine um aplicativo de um banco.

Quando o usuário informa a senha correta, o sistema deve permitir o acesso. Caso contrário, o acesso deve ser negado.

Ou pense em um sistema de uma loja. Se houver estoque disponível, a venda pode continuar. Caso contrário, o cliente deverá ser informado de que o produto está indisponível.

Percebe uma coisa?

O programa precisa **tomar uma decisão**.

É exatamente para isso que existem as **estruturas condicionais**.

Elas permitem que o programa escolha qual caminho seguir de acordo com uma condição. Em Python, utilizamos principalmente três estruturas para isso.

- if;
- elif;
- else.

Vamos conhecer cada uma delas.

IF

A palavra **if** significa "**se**".

Ela é utilizada quando queremos executar um bloco de código somente se determinada condição for verdadeira. Veja um exemplo.

```
idade = 20
```

```
if idade >= 18:
```

```
    print("Acesso permitido.")
```

Como a idade é maior que 18, o resultado será: Acesso permitido.

Agora imagine que a idade seja diferente.

```
idade = 15
```

```
if idade >= 18:
```

```
    print("Acesso permitido.")
```


Desta vez, nada será exibido. Isso acontece porque a condição não foi satisfeita. O Python simplesmente ignora aquele bloco de código.

É exatamente isso que o **if** faz. Ele pergunta:

"Essa condição é verdadeira?"

Se a resposta for **sim**, executa o código.

Se a resposta for **não**, continua normalmente.

A importância da indentação

Se existe uma característica que diferencia o Python de muitas outras linguagens, ela é a **indentação**.

Observe novamente o exemplo.

```
idade = 20
```

```
if idade >= 18:
```

```
    print("Acesso permitido.")
```

Repare que a linha do `print()` está um pouco mais para a direita. Esse espaço no início da linha recebe o nome de **indentação**.

No Python, ela não serve apenas para organizar o código. Ela faz parte da própria linguagem.

Em outras palavras. É a indentação que informa quais linhas pertencem ao bloco do **if**.

Se ela estiver incorreta, o programa poderá apresentar erro. Por isso, acostume-se desde o início a manter o código bem organizado.

Pense assim:

EU TENHO ESSA LINHA

TUDO QUE ESTÁ ESCRITO AQUI, FAZ PARTE DO “EU TENHO ESSA LINHA”

JÁ NESSA LINHA

TUDO QUE ESTÁ ESCRITO AQUI, FAZ PARTE DO “JÁ NESSA LINHA”

ELSE

Até agora vimos que o **if** executa um bloco quando a condição é verdadeira. Mas e

quando ela for falsa? É justamente para isso que existe o **else**.

A palavra **else** significa "**senão**". Veja um exemplo.

```
idade = 16
```

```
if idade >= 18:
```

```
    print("Acesso permitido.")
```

```
else:
```

```
    print("Acesso negado.")
```

Resultado: Acesso negado.

Agora altere a idade.

```
idade = 22
```

```
if idade >= 18:
```

```
    print("Acesso permitido.")
```

```
else:
```

```
    print("Acesso negado.")
```

Resultado: Acesso permitido.

Perceba que apenas um dos blocos será executado. Nunca os dois ao mesmo tempo.

ELIF

Imagine agora um sistema um pouco mais completo. Uma escola deseja classificar os alunos da seguinte forma.

- Nota maior ou igual a 7 → Aprovado;
- Nota entre 5 e 6,9 → Recuperação;
- Nota menor que 5 → Reprovado.

Perceba que agora temos mais de duas possibilidades. É justamente para isso que utilizamos o **elif**. A palavra **elif** significa algo como "**senão, se...**". Veja o exemplo.

```
nota = 6
```

```
if nota >= 7:
```

```
    print("Aprovado.")
```

```
elif nota >= 5:
```

```
    print("Recuperação.")
```

```
else:
```

```
    print("Reprovado.")
```

Resultado: Recuperação.

Observe como o programa analisa cada condição. Primeiro verifica:

A nota é maior ou igual a 7? Não.

Então passa para a próxima pergunta.

A nota é maior ou igual a 5? Sim.

Nesse momento ele executa o bloco correspondente e encerra a verificação.

Um exemplo do dia a dia

Imagine um aplicativo de entrega. Quando você acompanha um pedido, normalmente aparecem mensagens como:

1 Pedido recebido;

2 Pedido em preparação;

3 Saiu para entrega;

4 Entregue.

O sistema analisa a situação do pedido e exibe apenas a mensagem correspondente. Essa lógica é construída utilizando estruturas condicionais. Outro exemplo.

Quando você tenta acessar um sistema.

- Usuário e senha corretos → acesso liberado.
- Senha incorreta → acesso negado.
- Usuário bloqueado → contato com o suporte.

Tudo isso é decidido através de condições.

Podemos utilizar operadores lógicos

Lembra do assunto anterior?

Também podemos combinar condições utilizando **and**, **or** e **not**. Veja um exemplo.

```
idade = 25
```

```
cnh_valida = True
```

```
if idade >= 18 and cnh_valida:
```

```
    print("Aluguel autorizado.")
```

```
else:
```

```
    print("Aluguel não autorizado.")
```

Resultado: Aluguel autorizado.

Percebe como um assunto se conecta ao outro? É exatamente assim que os programas são construídos.

PROJETO PRÁTICO

Sistema de Aprovação de Crédito

Objetivo: desenvolver um programa capaz de analisar algumas informações do cliente e informar se um crédito foi aprovado.

Imagine que você foi contratado para desenvolver uma pequena parte de um sistema financeiro. Antes de aprovar um crédito, a empresa analisa algumas informações.

- Nome do cliente;
- Idade;
- Renda mensal;
- Valor solicitado.

Com base nesses dados, o sistema deverá informar se o crédito foi aprovado ou não. As regras serão as seguintes.

- O cliente deve possuir 18 anos ou mais;
- A renda deve ser igual ou superior a R\$ 3.000,00;
- O valor solicitado não poderá ultrapassar cinco vezes a renda mensal.

Caso todas as condições sejam atendidas, o crédito será aprovado. Caso contrário, o sistema deverá informar que o crédito foi negado.

Desafio

Desenvolva um programa que:

- Solicite o nome do cliente;
- Solicite a idade;
- Solicite a renda mensal;
- Solicite o valor desejado;
- Analise todas as condições;
- Exiba o resultado da análise.

Tente! Nada de ver a resposta direto.

Solução

Cadastro do cliente

```
nome = input("Digite seu nome: ")
```

```
idade = int(input("Digite sua idade: "))
```

```
renda = float(input("Digite sua renda mensal: R$ "))
```

```
credito = float(input("Digite o valor solicitado: R$ "))
```

Análise das condições

```
if idade >= 18 and renda >= 3000 and credito <= renda * 5:
```

```
    print()
```

```
    print("Parabéns,", nome + "!")
```

```
    print("Seu crédito foi aprovado.")
```

```
else:
```

```
    print()
```

```
    print("Infelizmente,", nome + ",")
```

```
    print("o crédito não foi aprovado.")
```

O que aprendemos neste projeto?

Durante este exercício utilizamos praticamente tudo o que vimos nesta apostila.

- Entrada de dados utilizando input();
- Exibição de informações com print();

- Conversão de tipos utilizando `int()` e `float()`;
- Operadores aritméticos para calcular o limite de crédito;
- Operadores relacionais para realizar comparações;
- Operador lógico `and` para combinar condições;
- Estrutura condicional `if` e `else` para tomar decisões.

Perceba que já somos capazes de desenvolver programas que recebem informações, processam dados e apresentam um resultado ao usuário.

Pode parecer um projeto simples. Mas é exatamente dessa forma que sistemas maiores começam a ser construídos.

Cada nova funcionalidade adiciona um pouco mais de inteligência ao programa.

Desafio Extra

Que tal deixar o sistema ainda mais completo? Bora?

Tente implementar as seguintes melhorias.

- Informar ao cliente o motivo da reprovação;
- Criar uma terceira situação utilizando `elif`, como "Crédito em análise";
- Calcular automaticamente o valor máximo que o cliente poderia solicitar;
- Permitir que o programa seja executado novamente sem precisar alterá-lo.

AGRADECIMENTO

Se você chegou até aqui, quero te dizer: obrigada.

De verdade.

Finalizar um material assim não é só sobre aprender conteúdo. É sobre dedicação, tempo e vontade de evoluir. E só de você estar aqui, já mostra muito sobre você.

Essa apostila foi criada com um propósito muito claro: mostrar que conhecimento não precisa ser complicado. Que assuntos que parecem difíceis podem, sim, ser entendidos quando explicados da forma certa.

E mais do que isso: que conhecimento deve ser para todos.

Por isso, este material também foi pensado para ser acessível. Ele conta com versão em Braille, porque inclusão não é um diferencial — é o mínimo.

Todo mundo merece aprender. Todo mundo merece ter acesso.

Se, de alguma forma, esse conteúdo te ajudou a entender melhor, a se organizar ou até a enxergar as coisas de um jeito diferente, então ele já cumpriu o seu papel.

Obrigada por fazer parte disso.

E lembre-se: você não precisa complicar para fazer dar certo.

CONTATO

Se você quiser continuar aprendendo, trocar ideias ou acompanhar mais conteúdos sobre projetos, tecnologia e organização no dia a dia, você pode me encontrar por aqui:

Instagram: @CAROL_G_INTECH

LinkedIn: Caroline Gonçalves

Site: <https://cpgconsulting.com.br/>

E-mail: cpgtechconsulting@gmail.com

Fique à vontade para se conectar ou mandar uma mensagem. Vou adorar continuar essa troca com você.